BÁO CÁO NGHIÊN CỨU HỌC THUẬT

Bài Toán Người Đi Du Lịch (TSP)

và Các Biến Thể

Tại Sao Chúng Ta Không Thể Tìm Ra Lời Giải Tối Ưu cho TSP Với Số Lượng Thành Phố Lớn Trong Thời Gian Thực?

Lĩnh vực: Khoa học Máy tính | Toán học Tổ hợp | Tối ưu hóa Tổ hợp

Phân loại: Lý thuyết Độ phức tạp | NP-Hard | Thuật toán Xấp xỉ

Sinh viên thực hiện: Trương Ngô Phát – 2045230075

Ngày biên soạn: Tháng 4 năm 2025

TÓM TẮT (ABSTRACT)

> *Bài toán Người đi du lịch (Travelling Salesman Problem - TSP) là một trong những bài toán tối ưu hóa tổ hợp nổi tiếng và được nghiên cứu nhiều nhất trong lịch sử khoa học máy tính và toán học. Với bề ngoài đơn giản — tìm hành trình ngắn nhất qua n thành phố, mỗi thành phố đúng một lần, rồi trở về điểm xuất phát — TSP ẩn chứa một độ phức tạp tính toán sâu sắc khiến nó trở thành biểu tượng của lớp bài toán NP-Hard.*
>
> **— Tổng quan báo cáo**

Báo cáo này trình bày một phân tích toàn diện về TSP, bao gồm: **(1)** định nghĩa hình thức và các biến thể chính của bài toán; **(2)** nền tảng lý thuyết độ phức tạp tính toán giải thích tại sao TSP không thể giải chính xác trong thời gian đa thức với kích thước lớn; **(3)** phân tích toán học về không gian giải pháp; **(4)** khảo sát các thuật toán chính xác, xấp xỉ và siêu heuristic; **(5)** ứng dụng thực tiễn; và **(6)** những hướng nghiên cứu mở đang được theo đuổi.

Qua phân tích kỹ lưỡng, báo cáo chứng minh rằng với n thành phố, số hành trình cần kiểm tra là (n-1)!/2, một con số tăng trưởng siêu hàm mũ khiến mọi thuật toán vét cạn đều trở nên bất khả thi. Các thuật toán tốt nhất hiện tại — như Held-Karp với độ phức tạp $O(n^2 \cdot 2^n)$ — chỉ giải được các trường hợp với vài chục đến vài trăm thành phố trong thời gian chấp nhận được. Trong khi đó, các bài toán thực tế thường có hàng nghìn đến hàng triệu điểm.

Chỉ số	Giá trị / Mô tả
Lớp độ phức tạp	NP-Hard (Tối ưu hóa), NP-Complete (Quyết định)
Số hành trình (n=20)	60,822,550,204,416,000 ≈ 6.1×10^{16}
Thuật toán chính xác tốt nhất	Held-Karp: $O(n^2 \cdot 2^n)$
Thuật toán xấp xỉ tốt nhất	Christofides: bảo đảm 3/2 tối ưu
Kỷ lục giải chính xác	85,900 thành phố (TSPLIB, 2006)
Ứng dụng thực tiễn tiêu biểu	Logistics, vi mạch, genome sequencing

Từ khóa: *Travelling Salesman Problem, NP-Hard, Lý thuyết độ phức tạp, Tối ưu hóa tổ hợp, Thuật toán xấp xỉ, Metaheuristic, Simulated Annealing, Genetic Algorithm, Branch and Bound, Held-Karp, Christofides Algorithm.*

MỤC LỤC

TÓM TẮT (ABSTRACT) 2
MỤC LỤC 3
CHƯƠNG 1: GIỚI THIỆU VÀ LỊCH SỬ PHÁT TRIỂN 9
1.1. Giới thiệu tổng quan 9
1.2. Lịch sử phát triển 9
1.2.1. Nguồn gốc và những nghiên cứu đầu tiên 9
1.2.2. Giai đoạn nền tảng (1930-1960) 10
1.2.3. Giai đoạn lý thuyết độ phức tạp (1971-1980) 10
1.2.4. Giai đoạn thuật toán siêu heuristic (1980-2000) 10
1.2.5. Giai đoạn học sâu và AI (2015-nay) 10
1.3. Tại sao TSP quan trọng? 11
CHƯƠNG 2: ĐỊNH NGHĨA HÌNH THỨC VÀ CÁC BIẾN THỂ 12
2.1. Định nghĩa hình thức 12
2.1.1. Biểu diễn đồ thị 12
2.1.2. Ma trận khoảng cách 12
2.1.3. Hai dạng bài toán 12
2.2. Phân loại TSP theo cấu trúc 13
2.2.1. TSP Đối xứng (Symmetric TSP - STSP) 13
2.2.2. TSP Bất đối xứng (Asymmetric TSP - ATSP) 13
2.2.3. TSP Metric (Metric TSP) 13
2.2.4. TSP Euclid (Euclidean TSP) 13
2.3. Các biến thể quan trọng của TSP 13
2.3.1. Capacitated Vehicle Routing Problem (CVRP) 13

2.3.2. TSP với Cửa sổ Thời gian (TSP with Time Windows - TSPTW) 14
2.3.3. Multiple Travelling Salesmen Problem (mTSP) 14
2.3.4. Orienteering Problem (OP) 14
2.3.5. TSP với Điểm Ưu tiên (Prize-Collecting TSP) 14
2.3.6. Generalized TSP (GTSP) 14
2.3.7. Stochastic TSP 15
2.3.8. Dynamic TSP 15
2.4. Bảng so sánh tổng hợp các biến thể 15
CHƯƠNG 3: LÝ THUYẾT ĐỘ PHỨC TẠP TÍNH TOÁN 16
3.1. Giới thiệu về lý thuyết độ phức tạp 16
3.2. Các lớp bài toán cơ bản 16
3.2.1. Lớp P — Polynomial Time 16
3.2.2. Lớp NP — Nondeterministic Polynomial Time 16
3.2.3. Lớp NP-Complete 17
3.2.4. Lớp NP-Hard 17
3.3. Sơ đồ quan hệ các lớp độ phức tạp 17
3.4. Chứng minh TSP là NP-Hard 18
3.4.1. Bước 1: TSP ∈ NP 18
3.4.2. Bước 2: TSP là NP-Hard (quy về từ Hamiltonian Cycle) 18
3.5. Hệ quả với thuật toán xấp xỉ 18
3.5.1. Định lý không xấp xỉ cho TSP tổng quát 18
CHƯƠNG 4: TẠI SAO KHÔNG THỂ GIẢI TSP TỐI ƯU VỚI SỐ LƯỢNG LỚN? .. 20
4.1. Phân tích không gian tìm kiếm 20
4.1.1. Số lượng hành trình có thể 20
4.1.2. Tốc độ tăng trưởng so sánh 20

4.2. Giới hạn của máy tính hiện đại 21
4.2.1. Tốc độ máy tính 21
4.2.2. Thuật toán Held-Karp — Giới hạn thực tế 21
4.3. Rào cản lý thuyết: Giả thuyết P ≠ NP 22
4.3.1. Phát biểu bài toán P vs NP 22
4.3.2. Tại sao chúng ta không thể chứng minh P ≠ NP? 22
4.3.3. Hệ quả thực tế 22
4.4. Rào cản kỹ thuật bổ sung 22
4.4.1. Bùng nổ không gian trạng thái 22
4.4.2. Phụ thuộc vào cấu trúc bài toán 23
4.4.3. Curse of Dimensionality 23
CHƯƠNG 5: CÁC THUẬT TOÁN CHÍNH XÁC 24
5.1. Vét cạn (Brute Force) 24
5.2. Thuật toán Held-Karp (Quy hoạch động) 24
5.2.1. Ý tưởng 24
5.2.2. Công thức quy hoạch động 24
5.2.3. Tại sao Held-Karp không giải được bài toán lớn? 25
5.3. Branch and Bound (Phân nhánh và Cắt) 25
5.3.1. Nguyên lý 25
5.3.2. Cận dưới: Lagrangian Relaxation và Assignment Problem 25
5.3.3. Hạn chế 25
5.4. Linear Programming và Cutting Plane 26
5.4.1. Công thức LP cho TSP 26
5.4.2. Cutting Plane Method 26
CHƯƠNG 6: THUẬT TOÁN XẤP XỈ VÀ HEURISTIC 27

6.1. Heuristic xây dựng (Construction Heuristics) 27
6.1.1. Nearest Neighbor Heuristic 27
6.1.2. Greedy Algorithm 27
6.1.3. Christofides Algorithm 27
6.2. Local Search — Tối ưu hóa cục bộ 28
6.2.1. 2-opt 28
6.2.2. 3-opt và k-opt 28
6.2.3. Lin-Kernighan Algorithm (LK) 28
6.3. Thuật toán Metaheuristic 28
6.3.1. Simulated Annealing (SA) 28
6.3.2. Genetic Algorithm (GA) 29
6.3.3. Ant Colony Optimization (ACO) 29
6.3.4. Particle Swarm Optimization (PSO) 29
6.3.5. Tabu Search (TS) 29
6.4. Neural Networks và Deep Learning 30
6.4.1. Pointer Networks (PtrNet) 30
6.4.2. Attention Model / Transformer-based 30
6.4.3. Graph Neural Networks (GNN) 30
6.5. So sánh tổng hợp các phương pháp 30
CHƯƠNG 7: ỨNG DỤNG THỰC TIỄN 32
7.1. Logistics và Giao thông vận tải 32
7.2. Sản xuất và Gia công 32
7.2.1. Thiết kế vi mạch điện tử (VLSI) 32
7.2.2. Lập lịch máy CNC 32
7.3. Sinh học và Y học 32

7.3.1. Giải trình tự DNA (DNA Sequencing) 32
7.3.2. Protein Folding 33
7.3.3. Lập lịch phẫu thuật 33
7.4. Thiên văn học và Khoa học 33
7.4.1. Quan sát kính thiên văn 33
7.4.2. Phân tích phổ (Spectroscopy) 33
7.5. Robotics và Tự động hóa 33
7.5.1. Coverage Path Planning 33
7.5.2. Pick-and-Place trong kho hàng 33
7.6. Mạng máy tính và Viễn thông 34
7.7. Tài chính 34
CHƯƠNG 8: HƯỚNG NGHIÊN CỨU MỞ VÀ TƯƠNG LAI 35
8.1. Các bài toán mở trong lý thuyết 35
8.1.1. Bài toán P vs NP 35
8.1.2. Cải thiện bảo đảm xấp xỉ 35
8.1.3. Cấu trúc bài toán khó nhất 35
8.2. Điện toán lượng tử và TSP 35
8.2.1. Thuật toán Grover 35
8.2.2. Quantum Annealing 35
8.2.3. QAOA (Quantum Approximate Optimization Algorithm) 36
8.3. Học máy và AI hiện đại 36
8.3.1. Large Language Models (LLM) và TSP 36
8.3.2. Neural Combinatorial Optimization (NCO) 36
8.3.3. Generalization Challenge 36
8.4. Hướng nghiên cứu ứng dụng 36

8.4.1. Real-time TSP 36
8.4.2. Multi-objective TSP 36
8.4.3. Green TSP 37
8.5. Tóm tắt hướng nghiên cứu 37
CHƯƠNG 9: KẾT LUẬN 38
9.1. Tổng hợp các phát hiện chính 38
9.1.1. Về bản chất của bài toán 38
9.1.2. Về trạng thái của thuật toán 38
9.1.3. Về tương lai 38
9.2. Trả lời câu hỏi trung tâm 38
9.3. Thông điệp cuối 39
TÀI LIỆU THAM KHẢO 40
Sách và Chuyên khảo 40
Bài báo Khoa học Quan trọng 40
Công cụ và Phần mềm 41
Tài nguyên Trực tuyến 41
PHỤ LỤC 42
Phụ lục A: Ví dụ tính toán Held-Karp 42
Phụ lục B: Bảng ký hiệu 43

CHƯƠNG 1: GIỚI THIỆU VÀ LỊCH SỬ PHÁT TRIỂN

1.1. Giới thiệu tổng quan

Bài toán Người đi du lịch (TSP) là một trong những bài toán tối ưu hóa tổ hợp nổi tiếng nhất trong lịch sử khoa học máy tính và toán học ứng dụng. Được phát biểu một cách đơn giản như sau: **"Cho n thành phố và khoảng cách giữa từng cặp thành phố, hãy tìm hành trình ngắn nhất đi qua tất cả các thành phố, mỗi thành phố đúng một lần, và trở về thành phố xuất phát."** Mặc dù phát biểu đơn giản, nhưng TSP ẩn chứa một độ phức tạp tính toán sâu sắc khiến nó không thể giải được một cách hiệu quả với kích thước lớn bằng bất kỳ thuật toán nào được biết đến hiện nay.

TSP không chỉ là một bài toán lý thuyết thuần túy — nó xuất hiện ở khắp nơi trong thực tiễn: lập lịch sản xuất, tối ưu tuyến đường giao hàng, thiết kế vi mạch điện tử, giải trình tự gene, lập kế hoạch robot, và nhiều lĩnh vực khác. Chính vì tính ứng dụng rộng rãi và độ khó lý thuyết của nó, TSP đã trở thành một trong những bài toán được nghiên cứu nhiều nhất trong suốt hơn 70 năm qua.

1.2. Lịch sử phát triển

1.2.1. Nguồn gốc và những nghiên cứu đầu tiên

Nguồn gốc của TSP có thể truy ngược về thế kỷ 19. Nhà toán học người Ireland William Rowan Hamilton và nhà toán học người Anh Thomas Kirkman đã nghiên cứu các bài toán liên quan đến chu trình Hamilton trong đồ thị vào những năm 1850-1860. Trò chơi Icosian Game của Hamilton (1857) yêu cầu tìm một chu trình đi qua 20 đỉnh của hình hai mươi mặt, là một tiền thân trực tiếp của TSP.

Năm	Sự kiện / Nghiên cứu	Ý nghĩa
1832	Sách hướng dẫn cho người đi du lịch xuất bản ở Đức	Đề cập đến vấn đề tìm hành trình hiệu quả
1857	Hamilton giới thiệu Icosian Game	Tiền thân trực tiếp của chu trình Hamilton
1930	Karl Menger phát biểu bài toán Botenproblem (messenger problem)	Phát biểu hình thức đầu tiên gần với TSP hiện đại
1954	Dantzig, Fulkerson, Johnson giải TSP 49 thành phố	Kỷ lục đầu tiên, sử dụng quy hoạch tuyến tính
1972	Karp chứng minh TSP là NP-Complete	Bước ngoặt lý thuyết quan trọng nhất
1976	Christofides công bố thuật toán xấp xỉ 3/2	Bảo đảm lý thuyết tốt nhất cho metric TSP

1991	Lin-Kernighan-Helsgott (LKH) - công cụ thực tế mạnh nhất	Giải bài toán thực tế hiệu quả
2006	Applegate et al. giải TSP 85,900 thành phố (Sweden)	Kỷ lục thế giới về TSP chính xác
2012-nay	Deep Learning và RL áp dụng vào TSP	Hướng nghiên cứu mới đầy tiềm năng

Bảng 1.1: Mốc lịch sử quan trọng trong nghiên cứu TSP

1.2.2. Giai đoạn nền tảng (1930-1960)

Năm 1930, nhà toán học người Áo Karl Menger lần đầu tiên phát biểu bài toán dưới tên 'Botenproblem' (bài toán người đưa tin). Merrill Flood tại RAND Corporation đã phổ biến bài toán này trong cộng đồng nghiên cứu Hoa Kỳ vào đầu những năm 1950 và đặt tên là 'Travelling Salesman Problem'. Năm 1954, George Dantzig, Ray Fulkerson và Selmer Johnson đã giải được TSP với 49 thành phố đại diện cho các thành phố của Hoa Kỳ bằng phương pháp quy hoạch tuyến tính kết hợp với phương pháp cắt mặt phẳng — một thành tựu đáng kinh ngạc vào thời điểm đó.

1.2.3. Giai đoạn lý thuyết độ phức tạp (1971-1980)

Giai đoạn này đánh dấu bước ngoặt lý thuyết quan trọng nhất. Năm 1971, Stephen Cook chứng minh định lý Cook-Levin, thiết lập lớp NP-Complete. Năm 1972, Richard Karp chứng minh rằng bài toán quyết định của TSP (tức là: có tồn tại hành trình với tổng khoảng cách nhỏ hơn hoặc bằng k không?) là NP-Complete. Điều này dẫn đến một hệ quả cực kỳ quan trọng: nếu $P \neq NP$ (điều mà hầu hết các nhà khoa học đều tin), thì **không tồn tại thuật toán đa thức nào có thể luôn tìm ra hành trình tối ưu cho mọi trường hợp TSP.** Christofides (1976) công bố thuật toán xấp xỉ nổi tiếng với bảo đảm 3/2 — kết quả lý thuyết tốt nhất cho đến nay cho metric TSP.

1.2.4. Giai đoạn thuật toán siêu heuristic (1980-2000)

Giai đoạn này chứng kiến sự bùng nổ của các phương pháp metaheuristic: Simulated Annealing (Kirkpatrick et al., 1983), Genetic Algorithm (Holland, áp dụng vào TSP bởi nhiều nhóm), Tabu Search (Glover, 1986), Ant Colony Optimization (Dorigo, 1992), và Particle Swarm Optimization (Kennedy & Eberhart, 1995). Những phương pháp này không đảm bảo tìm ra nghiệm tối ưu nhưng thường cho kết quả rất tốt trong thực tế với thời gian tính toán chấp nhận được.

1.2.5. Giai đoạn học sâu và AI (2015-nay)

Kỷ nguyên học sâu mở ra hướng tiếp cận mới đầy hứa hẹn. Năm 2015, Vinyals et al. giới thiệu Pointer Networks, một kiến trúc deep learning đặc biệt cho TSP. Năm 2017, Bello et al. áp dụng Reinforcement Learning để giải TSP. Năm 2019, Kool et al. đề xuất Attention Model vượt trội hơn nhiều phương pháp truyền thống cho TSP. Năm 2020-2022, các mô hình Graph Neural Network (GNN) và Transformer tiếp tục đẩy xa ranh giới. Tuy nhiên, các phương pháp AI hiện tại vẫn chưa thể vượt qua các heuristic tốt nhất cho bài toán lớn.

1.3. Tại sao TSP quan trọng?

TSP quan trọng vì ba lý do cơ bản:

- **Tính ứng dụng thực tiễn rộng rãi:** TSP và các biến thể của nó xuất hiện trong hàng trăm bài toán thực tế thuộc nhiều ngành công nghiệp khác nhau, từ logistics đến sinh tin học.
- **Giá trị lý thuyết sâu sắc:** TSP là nguyên mẫu hoàn hảo của lớp NP-Hard — bài toán dễ phát biểu nhưng vô cùng khó giải. Nghiên cứu TSP đã thúc đẩy sự phát triển của lý thuyết độ phức tạp, thuật toán xấp xỉ, và tối ưu hóa tổ hợp.
- **Kết nối với câu hỏi P vs NP:** Một thuật toán đa thức cho TSP sẽ giải quyết (hoặc chỉ ra con đường giải quyết) bài toán P vs NP — một trong bảy Bài toán Thiên niên kỷ của Viện Toán học Clay, với phần thưởng 1 triệu đô la Mỹ.

CHƯƠNG 2: ĐỊNH NGHĨA HÌNH THỨC VÀ CÁC BIẾN THỂ

2.1. Định nghĩa hình thức

2.1.1. Biểu diễn đồ thị

Xét đồ thị đầy đủ có trọng số G = (V, E, w) trong đó:

- $V = \{v_1, v_2, ..., v_n\}$ là tập hợp n đỉnh (thành phố)
- $E = \{(v_i, v_j) : i \neq j\}$ là tập hợp các cạnh (khoảng cách giữa các cặp thành phố)
- $w : E \rightarrow \mathbb{R}^+$ là hàm trọng số (khoảng cách)

Một **chu trình Hamilton** là một hoán vị $\pi = (\pi(1), \pi(2), ..., \pi(n))$ của tập đỉnh V. Tổng độ dài của chu trình là:

$$L(\pi) = \Sigma_{i=1}^{n-1} w(\pi(i), \pi(i+1)) + w(\pi(n), \pi(1))$$

Mục tiêu TSP là tìm hoán vị tối ưu:

$$\pi^* = \text{argmin } L(\pi) = \text{argmin } \{ \Sigma_{i=1}^{n-1} w(\pi(i), \pi(i+1)) + w(\pi(n), \pi(1)) \}$$

2.1.2. Ma trận khoảng cách

TSP thường được biểu diễn bằng ma trận khoảng cách $D = [d_{ij}]_{nxn}$ trong đó d_{ij} là khoảng cách từ thành phố i đến thành phố j. Ma trận này có thể đối xứng ($d_{ij} = d_{ji}$ — TSP đối xứng) hoặc bất đối xứng ($d_{ij} \neq d_{ji}$ — TSP bất đối xứng). Ví dụ với n = 4 thành phố:

$$D = \begin{bmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{bmatrix}$$

Ma trận khoảng cách đối xứng ví dụ với 4 thành phố

2.1.3. Hai dạng bài toán

TSP có hai dạng chính:

- **Bài toán quyết định (Decision Problem):** Cho đồ thị G và ngưỡng k, có tồn tại chu trình Hamilton với tổng trọng số ≤ k không? → Đây là bài toán NP-Complete (chứng minh bởi Karp, 1972).
- **Bài toán tối ưu hóa (Optimization Problem):** Tìm chu trình Hamilton có tổng trọng số nhỏ nhất. → Đây là bài toán NP-Hard (khó ít nhất bằng dạng quyết định và thậm chí còn khó hơn).

2.2. Phân loại TSP theo cấu trúc

2.2.1. TSP Đối xứng (Symmetric TSP - STSP)

Trong STSP, khoảng cách giữa hai thành phố i và j là như nhau theo cả hai chiều: $d_{ij} = d_{ji}$. Đây là dạng phổ biến nhất trong thực tế (đường bộ thông thường, khoảng cách Euclid). Do tính đối xứng, số hành trình phân biệt là (n-1)!/2 thay vì (n-1)!.

2.2.2. TSP Bất đối xứng (Asymmetric TSP - ATSP)

Trong ATSP, $d_{ij} \neq d_{ji}$ — tức là khoảng cách hoặc chi phí từ i đến j khác với từ j đến i. Điều này xảy ra trong nhiều tình huống thực tế: đường một chiều, chi phí vận chuyển phụ thuộc hướng, mạng giao thông có đường cao tốc, sông ngòi, v.v. ATSP tổng quát hơn và khó hơn STSP.

2.2.3. TSP Metric (Metric TSP)

TSP Metric yêu cầu bất đẳng thức tam giác được thỏa mãn: $d_{ij} \leq d_{ik} + d_{kj}$ cho mọi i, j, k. Đây là trường hợp quan trọng vì hầu hết bài toán thực tế thỏa mãn điều kiện này (khoảng cách Euclid, khoảng cách trên mặt cầu, v.v.). Điều kiện metric là tiền đề để các thuật toán xấp xỉ có bảo đảm lý thuyết hoạt động.

2.2.4. TSP Euclid (Euclidean TSP)

Các thành phố nằm trên mặt phẳng Euclid với tọa độ (x_i, y_i) và khoảng cách:

$$d_{ij} = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

Euclidean TSP có cấu trúc hình học đặc biệt cho phép khai thác trong nhiều thuật toán. Đây là dạng được nghiên cứu nhiều nhất và có nhiều ứng dụng nhất.

2.3. Các biến thể quan trọng của TSP

2.3.1. Capacitated Vehicle Routing Problem (CVRP)

CVRP là tổng quát hóa quan trọng nhất của TSP trong logistics. Thay vì một người đi du lịch, ta có **k xe tải** xuất phát từ một kho hàng (depot), mỗi xe có sức chứa giới hạn C. Mỗi khách hàng có nhu cầu d_i. Mục tiêu: phân chia khách hàng vào các tuyến đường sao cho tổng khoảng cách là nhỏ nhất, không vi phạm sức chứa.

Công thức ràng buộc CVRP
Tổng nhu cầu mỗi tuyến: $\Sigma_i \in R_k\ d_i \leq C$
Mỗi khách hàng thuộc đúng một tuyến: Mỗi $i \in V\backslash\{depot\}$ thuộc đúng một R_k
Tối thiểu: $\Sigma_k\ L(R_k)$ trong đó L là độ dài tuyến đường

2.3.2. TSP với Cửa sổ Thời gian (TSP with Time Windows - TSPTW)

TSPTW thêm ràng buộc thời gian: mỗi thành phố i phải được thăm trong khoảng thời gian $[a_i, b_i]$ (cửa sổ thời gian). Nếu đến sớm hơn a_i, người giao hàng phải chờ. Nếu đến muộn hơn b_i, vi phạm ràng buộc. Bài toán này xuất hiện trong giao hàng nhanh, lịch trình bệnh viện, và logistics thời gian thực.

2.3.3. Multiple Travelling Salesmen Problem (mTSP)

mTSP có m người đi du lịch cùng xuất phát từ một depot. Bài toán yêu cầu phân chia các thành phố cho m người sao cho tổng khoảng cách (hoặc khoảng cách dài nhất) là nhỏ nhất. mTSP là tiền thân tự nhiên của VRP và có nhiều ứng dụng trong robotics và quản lý nhân lực.

2.3.4. Orienteering Problem (OP)

OP là biến thể trong đó mỗi thành phố có một điểm thưởng s_i và người đi du lịch bị giới hạn tổng khoảng cách (hoặc thời gian) là B. Mục tiêu không phải đi qua tất cả thành phố mà tối đa hóa tổng điểm thưởng thu thập được. OP áp dụng trong lập kế hoạch tuyến du lịch, trinh sát quân sự, và thu thập dữ liệu sensor.

2.3.5. TSP với Điểm Ưu tiên (Prize-Collecting TSP)

Trong PC-TSP, người đi du lịch không bắt buộc phải ghé thăm tất cả các thành phố. Bỏ qua thành phố i sẽ phải trả phạt π_i. Mục tiêu là tối thiểu hóa tổng khoảng cách cộng với tổng tiền phạt cho các thành phố bị bỏ qua.

2.3.6. Generalized TSP (GTSP)

GTSP chia các thành phố thành các nhóm (cluster). Người đi du lịch phải ghé thăm đúng một thành phố từ mỗi nhóm và tối thiểu hóa tổng khoảng cách. GTSP xuất hiện trong bài toán lựa chọn nhà cung cấp, lập kế hoạch đường bay với nhiều sân bay cho mỗi điểm đến.

2.3.7. Stochastic TSP

Trong Stochastic TSP, các thành phố hoặc khoảng cách không cố định mà ngẫu nhiên. Ví dụ: khách hàng có thể có mặt hoặc vắng mặt với xác suất nhất định, hoặc thời gian di chuyển có thể biến thiên theo điều kiện giao thông. Bài toán này rất quan trọng trong logistics thực tế.

2.3.8. Dynamic TSP

Dynamic TSP (DTSP) đối mặt với thay đổi theo thời gian thực: các thành phố mới xuất hiện, điểm đến bị hủy, hoặc điều kiện đường xá thay đổi. Người đi du lịch phải thích ứng và cập nhật kế hoạch liên tục. DTSP rất quan trọng trong các hệ thống giao hàng theo yêu cầu (on-demand delivery).

2.4. Bảng so sánh tổng hợp các biến thể

Biến thể	**Đặc điểm chính**	**Độ khó**	**Ứng dụng tiêu biểu**
TSP đối xứng	$d_{ij} = d_{ji}$	NP-Hard	Lập lịch, logistics
TSP bất đối xứng	$d_{ij} \neq d_{ji}$	NP-Hard	Giao thông một chiều
CVRP	Nhiều xe, sức chứa giới hạn	NP-Hard	Phân phối hàng hóa
TSPTW	Cửa sổ thời gian	NP-Hard	Giao hàng đúng giờ
mTSP	Nhiều người đi du lịch	NP-Hard	Quản lý nhân lực
Orienteering	Giới hạn ngân sách, điểm thưởng	NP-Hard	Du lịch, trinh sát
GTSP	Chọn 1 thành phố mỗi nhóm	NP-Hard	Lựa chọn nhà cung cấp
Stochastic TSP	Tham số ngẫu nhiên	NP-Hard+	Logistics thực tế
Dynamic TSP	Thay đổi theo thời gian	NP-Hard+	Giao hàng theo yêu cầu

Bảng 2.1: So sánh các biến thể của TSP

CHƯƠNG 3: LÝ THUYẾT ĐỘ PHỨC TẠP TÍNH TOÁN

3.1. Giới thiệu về lý thuyết độ phức tạp

Lý thuyết độ phức tạp tính toán (Computational Complexity Theory) là ngành nghiên cứu về các tài nguyên tính toán — thời gian và không gian bộ nhớ — cần thiết để giải các bài toán. Lý thuyết này cho phép chúng ta phân loại các bài toán theo mức độ khó và trả lời câu hỏi căn bản: 'Bài toán này có thể được giải hiệu quả không?'

3.2. Các lớp bài toán cơ bản

3.2.1. Lớp P — Polynomial Time

Lớp P (Polynomial) gồm tất cả các bài toán quyết định có thể được giải bằng thuật toán xác định (deterministic algorithm) trong thời gian **đa thức** theo kích thước đầu vào. Nói cách khác, bài toán với kích thước đầu vào n có thể giải trong thời gian $O(n^k)$ với k là hằng số.

Ví dụ bài toán thuộc P: sắp xếp (O(n log n)), tìm đường ngắn nhất — Dijkstra ($O(n^2)$), nhân ma trận ($O(n^3)$), kiểm tra số nguyên tố (AKS, đa thức), bài toán luồng mạng tối đa.

3.2.2. Lớp NP — Nondeterministic Polynomial Time

Lớp NP gồm các bài toán quyết định mà một lời giải đề xuất (certificate) có thể được **xác minh** trong thời gian đa thức. Cần phân biệt: NP không có nghĩa là 'Non-Polynomial' — mà là 'Nondeterministic Polynomial'. Mọi bài toán P đều thuộc NP (vì giải được = xác minh được), nhưng chưa biết NP có bằng P không.

Ví dụ minh họa: Xác minh nghiệm TSP
Câu hỏi: Có tồn tại hành trình ≤ k không? Giả sử ai đó đưa cho bạn một hành trình $\pi = (\pi_1, \pi_2, ..., \pi_n, \pi_1)$ Xác minh: Tính $L(\pi) = \Sigma\, w(\pi_i, \pi_{i+1})$, kiểm tra $L(\pi) \leq k \rightarrow O(n)$ = đa thức ✓ **→ TSP Decision ∈ NP (xác minh nghiệm dễ, tìm nghiệm khó)**

3.2.3. Lớp NP-Complete

Một bài toán L là NP-Complete nếu:

1. **L ∈ NP:** nghiệm có thể xác minh trong thời gian đa thức
2. **L là NP-Hard:** mọi bài toán trong NP có thể quy về L trong thời gian đa thức (polynomial reduction). Tức là L là 'khó ít nhất' bằng mọi bài toán NP.

Ví dụ NP-Complete: SAT (Boolean Satisfiability), 3-SAT, Vertex Cover, Clique, Independent Set, Graph Coloring, Subset Sum, Knapsack (dạng quyết định), và tất nhiên là **TSP (dạng quyết định).**

3.2.4. Lớp NP-Hard

NP-Hard là lớp các bài toán 'ít nhất khó như NP-Complete' nhưng không nhất thiết phải thuộc NP. Một bài toán L là NP-Hard nếu mọi bài toán NP-Complete có thể quy về L trong thời gian đa thức. NP-Hard bao gồm cả bài toán tối ưu hóa không phải dạng quyết định.

TSP tối ưu hóa (Optimization TSP) là NP-Hard vì: (1) Nếu ta giải được TSP tối ưu hóa, ta giải được TSP quyết định. (2) TSP quyết định là NP-Complete. (3) Do đó TSP tối ưu hóa ít nhất khó bằng một bài toán NP-Complete, tức là NP-Hard.

3.3. Sơ đồ quan hệ các lớp độ phức tạp

Quan hệ giữa các lớp (giả sử P ≠ NP)
NP-Hard (bao gồm các bài toán 'ngoài NP') ↑ bao gồm **NP-Complete (giao điểm NP ∩ NP-Hard)** ↑ thuộc **NP (xác minh đa thức)** ↑ thuộc **P (giải đa thức) ⊆ NP** *TSP thuộc NP-Complete (dạng quyết định) và NP-Hard (dạng tối ưu hóa)*

3.4. Chứng minh TSP là NP-Hard

3.4.1. Bước 1: TSP ∈ NP

Cho một chu trình Hamilton $\pi = (\pi(1), \pi(2), ..., \pi(n))$, ta có thể xác minh trong thời gian O(n): kiểm tra π là hoán vị của {1,...,n}, tính $L(\pi) = \Sigma\, d_{\pi(i),\pi(i+1)} + d_{\pi(n),\pi(1)}$, và kiểm tra $L(\pi) \leq k$. Do đó TSP ∈ NP. ✓

3.4.2. Bước 2: TSP là NP-Hard (quy về từ Hamiltonian Cycle)

Ta chứng minh **Hamiltonian Cycle $\leq_p$ TSP** (quy đổi đa thức).

Đầu vào: Đồ thị G = (V, E) với |V| = n.

Xây dựng đồ thị TSP: Tạo đồ thị TSP với n thành phố và ma trận khoảng cách:

$$d_{ij} = 0 \text{ nếu } (i,j) \in E \;\mid\; d_{ij} = 1 \text{ nếu } (i,j) \notin E$$

Tuyên bố: G có chu trình Hamilton ⟺ TSP tương ứng có hành trình với độ dài ≤ 0.

Chứng minh (⟹): Nếu G có chu trình Hamilton C, thì C cũng là một hành trình hợp lệ trong TSP. Vì C chỉ đi qua các cạnh có trong E (trọng số 0), nên L(C) = 0 ≤ 0. ✓

Chứng minh (⟸): Nếu TSP có hành trình π với $L(\pi) \leq 0$, vì mọi trọng số ≥ 0, ta phải có $L(\pi) = 0$. Điều này nghĩa là mọi cạnh trong π đều có trọng số 0, tức là thuộc E. Do đó π là chu trình Hamilton trong G. ✓

Kết luận: Vì Hamiltonian Cycle là NP-Complete và có thể quy về TSP trong thời gian đa thức, TSP là NP-Hard. Kết hợp với TSP ∈ NP, ta suy ra TSP là NP-Complete (dạng quyết định).

3.5. Hệ quả với thuật toán xấp xỉ

3.5.1. Định lý không xấp xỉ cho TSP tổng quát

Định lý (Sahni & Gonzalez, 1976): Nếu P ≠ NP, thì không tồn tại thuật toán đa thức có hệ số xấp xỉ hữu hạn cho TSP tổng quát (không có bất đẳng thức tam giác).

Chứng minh (phác thảo): Giả sử tồn tại thuật toán A cho TSP với hệ số xấp xỉ r. Ta dùng A để giải Hamiltonian Cycle như sau: Với đồ thị G, xây dựng TSP như trên nhưng với $d_{ij} = 0$ nếu $(i,j) \in E$ và $d_{ij} = r \cdot n + 1$ nếu $(i,j) \notin E$. Nếu G có chu trình Hamilton, hành trình tối ưu có độ dài 0. Nếu không, mọi hành trình có ít nhất một cạnh không thuộc E, nên độ dài $\geq r \cdot n+1$. Thuật toán A với hệ số r trả về hành trình ≤

$r \cdot 0 = 0$ chỉ khi hành trình tối ưu là 0, tức khi G có chu trình Hamilton. Điều này giải Hamiltonian Cycle trong đa thức — mâu thuẫn với $P \neq NP$.

Hệ quả thực tiễn: Đây là lý do tại sao chúng ta CHỈ có thể xây dựng thuật toán xấp xỉ cho TSP Metric (với bất đẳng thức tam giác), không thể cho TSP tổng quát. Đối với metric TSP, thuật toán Christofides bảo đảm tỉ lệ 3/2.

CHƯƠNG 4: TẠI SAO KHÔNG THỂ GIẢI TSP TỐI ƯU VỚI SỐ LƯỢNG LỚN?

4.1. Phân tích không gian tìm kiếm

4.1.1. Số lượng hành trình có thể

Với n thành phố, tổng số chu trình Hamilton (tính có hướng) là (n-1)!. Với TSP đối xứng (không phân biệt chiều), số hành trình phân biệt là (n-1)!/2. Bảng sau minh họa sự tăng trưởng bùng nổ tổ hợp (combinatorial explosion):

n (thành phố)	Số hành trình: (n-1)!/2	So sánh
5	12	Có thể duyệt bằng tay
10	181,440	$\approx 2 \times 10^5$
15	43,589,145,600	$\approx 4.4 \times 10^{10}$
20	60,822,550,204,416,000	$\approx 6.1 \times 10^{16}$
25	$\approx 3.1 \times 10^{23}$	Nhiều hơn số phân tử nước trong biển
50	$\approx 1.5 \times 10^{63}$	Nhiều hơn số nguyên tử trong vũ trụ quan sát được
100	$\approx 4.7 \times 10^{157}$	Hoàn toàn không thể liệt kê
1,000	$\approx 10^{2,567}$	Bất khả thi tuyệt đối

Bảng 4.1: Số lượng hành trình theo số thành phố

So sánh với quy mô vũ trụ
• Số nguyên tử trong vũ trụ quan sát được: $\approx 10^{80}$ • Tuổi vũ trụ tính bằng nano-giây: $\approx 4.3 \times 10^{26}$ ns • Số hành trình TSP với n=62: $\approx 10^{84}$ (nhiều hơn số nguyên tử trong vũ trụ!) **→ Vét cạn tất cả hành trình với n=62 là vật lý bất khả thi, kể cả khi dùng tất cả nguyên tử vũ trụ làm máy tính.**

4.1.2. Tốc độ tăng trưởng so sánh

Để hiểu tại sao kích thước bài toán lớn là nan giải, hãy so sánh các tốc độ tăng trưởng:

Loại tăng trưởng	Công thức	n=10	n=20	n=50	n=100

Tuyến tính	n	10	20	50	100
Bậc hai	n^2	100	400	2,500	10,000
Đa thức	n^{10}	10^{10}	10^{13}	10^{17}	10^{20}
Hàm mũ (2^n)	2^n	1,024	10^6	10^{15}	10^{30}
Held-Karp ($n^2 \cdot 2^n$)	$n^2 \cdot 2^n$	$\sim 10^5$	$\sim 10^8$	$\sim 10^{17}$	$\sim 10^{32}$
Giai thừa ((n-1)!/2)	(n-1)!/2	181K	6×10^{16}	1.5×10^{63}	4.7×10^{157}

Bảng 4.2: So sánh tốc độ tăng trưởng các hàm

4.2. Giới hạn của máy tính hiện đại

4.2.1. Tốc độ máy tính

Máy tính nhanh nhất hiện nay (siêu máy tính Frontier tại ORNL, 2023) đạt khoảng **1.1 ExaFLOPS = 1.1 × 10^{18}** phép tính dấu phẩy động mỗi giây. Tuy nhiên, con số khổng lồ này vẫn hoàn toàn vô nghĩa trước sự tăng trưởng giai thừa:

n	Số hành trình	Thời gian @ 10^{18} ops/s	Kết luận
20	6.1×10^{16}	~0.06 giây	Khả thi (nhưng vét cạn thô)
25	3.1×10^{23}	~9.8 triệu năm	Không thực tế
30	$\approx 4.4 \times 10^{30}$	$\sim 1.4 \times 10^5$ tỷ năm	Bất khả thi tuyệt đối
50	1.5×10^{63}	$\sim 1.5 \times 10^{36}$ tỷ năm	Vô nghĩa
100	4.7×10^{157}	$\sim 5 \times 10^{130}$ tỷ năm	Hoàn toàn không tưởng

Bảng 4.3: Ước tính thời gian vét cạn với siêu máy tính

4.2.2. Thuật toán Held-Karp — Giới hạn thực tế

Thuật toán Held-Karp (1962) là thuật toán chính xác tốt nhất được biết đến cho TSP, với độ phức tạp $O(n^2 \cdot 2^n)$ về thời gian và $O(n \cdot 2^n)$ về không gian. Dù vượt trội hơn vét cạn thô (O(n!)), Held-Karp vẫn bị hạn chế bởi bùng nổ hàm mũ:

n	Thời gian (Held-Karp) @ 10^9 ops/s	Bộ nhớ
20	~4 ms	~20 MB
30	~1.1 giây	~32 GB
40	~18 phút	~44 TB
50	~3,200 giờ (~4 tháng)	~56,000 TB
100	Khoảng vũ trụ nguyên tử	Hoàn toàn không khả thi

Bảng 4.4: Giới hạn thực tế của thuật toán Held-Karp

Trên thực tế, Held-Karp chỉ khả thi với khoảng **n ≤ 20-25** trên máy tính cá nhân, và khoảng **n ≤ 30-35** trên siêu máy tính với bộ nhớ lớn.

4.3. Rào cản lý thuyết: Giả thuyết P ≠ NP

4.3.1. Phát biểu bài toán P vs NP

Câu hỏi P vs NP là bài toán mở quan trọng nhất trong khoa học máy tính lý thuyết: **"Liệu P = NP hay P ≠ NP?"** Nếu P = NP, thì mọi bài toán mà nghiệm có thể xác minh nhanh cũng có thể được tìm nhanh — bao gồm cả TSP, mã hóa RSA có thể bị phá, AI có thể giải mọi bài toán logic, v.v. Nếu P ≠ NP (điều hầu hết nhà khoa học tin), thì TSP về cơ bản là 'không thể giải hiệu quả'.

4.3.2. Tại sao chúng ta không thể chứng minh P ≠ NP?

Đây là lý do bài toán P vs NP vẫn chưa được giải sau hơn 50 năm. Về cơ bản, chứng minh rằng một vấn đề không có thuật toán hiệu quả đòi hỏi lập luận về **tất cả** các thuật toán có thể — kể cả những thuật toán chưa được phát minh. Các rào cản kỹ thuật đã được xác định gồm: **Relativization** (Baker-Gill-Solovay, 1975), **Natural Proofs** (Razborov-Rudich, 1994), và **Algebrization** (Aaronson-Wigderson, 2009). Mỗi rào cản này loại bỏ một lớp lớn các phương pháp chứng minh tiềm năng.

4.3.3. Hệ quả thực tế

Dù P vs NP chưa được giải, cộng đồng khoa học đã đi đến một đồng thuận thực tế: chúng ta thiết kế hệ thống và thuật toán với giả định P ≠ NP. Điều này có nghĩa là:

- **Mã hóa học hiện đại** (RSA, AES, ECC) dựa trên giả định NP-Hard không thể phá được
- **TSP và các bài toán NP-Hard** được giải bằng thuật toán xấp xỉ và heuristic, không phải thuật toán chính xác đa thức
- **Thiết kế hệ thống thực tế** chấp nhận nghiệm 'đủ tốt' thay vì nghiệm tối ưu tuyệt đối

4.4. Rào cản kỹ thuật bổ sung

4.4.1. Bùng nổ không gian trạng thái

Ngay cả với thuật toán thông minh như Branch and Bound hoặc Held-Karp, không gian trạng thái (state space) vẫn tăng hàm mũ. Trong Branch and Bound, cây tìm kiếm có thể có O(n!) nút trong trường hợp xấu nhất. Kỹ thuật cắt tỉa (pruning) giảm đáng kể nhưng không thể đảm bảo thời gian đa thức.

4.4.2. Phụ thuộc vào cấu trúc bài toán

Một số bài toán TSP có cấu trúc đặc biệt (ví dụ: thành phố nằm trên đường thẳng, hoặc có cấu trúc nhóm rõ ràng) và có thể giải nhanh hơn nhiều. Nhưng TSP tổng quát, đặc biệt là khi khoảng cách ngẫu nhiên hoặc không có cấu trúc, không cho phép khai thác đặc điểm này.

4.4.3. Curse of Dimensionality

Khi n tăng, không chỉ không gian giải pháp tăng mà còn xuất hiện 'lời nguyền chiều số' (curse of dimensionality). Các thuật toán tìm kiếm cục bộ (local search) như 2-opt, 3-opt dễ bị kẹt ở điểm tối ưu cục bộ (local optima) trong các cảnh quan giải pháp phức tạp. Số lượng điểm tối ưu cục bộ cũng tăng theo n.

CHƯƠNG 5: CÁC THUẬT TOÁN CHÍNH XÁC

5.1. Vét cạn (Brute Force)

Phương pháp đơn giản nhất: liệt kê tất cả (n-1)! hoán vị, tính độ dài mỗi hành trình, và chọn hành trình ngắn nhất.

Pseudocode: Brute Force TSP

function BruteForce_TSP(D[1..n][1..n]):
  best_cost ← ∞; best_tour ← null
  for each permutation π of {2, 3, ..., n}:
  tour ← [1] + π + [1] // fix city 1 as start
  cost ← Σ D[tour[i]][tour[i+1]]
  if cost < best_cost:
  best_cost ← cost; best_tour ← tour
  return best_tour, best_cost
Độ phức tạp: Thời gian O(n!), Không gian O(n)

5.2. Thuật toán Held-Karp (Quy hoạch động)

5.2.1. Ý tưởng

Held-Karp (1962, độc lập bởi Bellman, Held và Karp) là thuật toán quy hoạch động (dynamic programming) đầu tiên giải TSP tốt hơn vét cạn. Ý tưởng cốt lõi: thay vì liệt kê tất cả hành trình, ta dùng lại (reuse) kết quả của các bài toán con (subproblems).

5.2.2. Công thức quy hoạch động

Định nghĩa: C(S, i) = độ dài đường đi ngắn nhất bắt đầu từ thành phố 1, đi qua đúng tập thành phố S, và kết thúc tại thành phố i ∈ S.

Trường hợp cơ sở: $C(\{i\}, i) = d_{1i}$ (chỉ thăm thành phố i từ thành phố 1)

Bước đệ quy: $C(S, i) = \min_{j \in S \setminus \{i\}} [C(S \setminus \{i\}, j) + d_{ji}]$

Nghiệm tối ưu: $OPT = \min_{i \neq 1} [C(V, i) + d_{i1}]$

Số trạng thái cần tính: $O(n \cdot 2^n)$ (n lựa chọn điểm kết thúc × 2^n tập con). Mỗi trạng thái tốn $O(n)$ để tính. Tổng: $O(n^2 \cdot 2^n)$ thời gian, $O(n \cdot 2^n)$ không gian.

5.2.3. Tại sao Held-Karp không giải được bài toán lớn?

Với n = 40: bộ nhớ cần ≈ $40 \times 2^{40} \times 8$ bytes ≈ 350 TB — vượt quá RAM của bất kỳ siêu máy tính nào. Với n = 30: bộ nhớ ≈ 32 GB — có thể thực hiện nhưng cần nhiều giờ. Do đó, Held-Karp trong thực tế chỉ giải được $n \leq 25$-30.

5.3. Branch and Bound (Phân nhánh và Cắt)

5.3.1. Nguyên lý

Branch and Bound (B&B) là phương pháp tìm kiếm có hệ thống trong không gian giải pháp, kết hợp:

- **Branching (Phân nhánh):** Chia bài toán thành các bài toán con (cây tìm kiếm)
- **Bounding (Giới hạn):** Tính cận dưới (lower bound) cho mỗi nhánh
- **Pruning (Cắt tỉa):** Loại bỏ nhánh nếu cận dưới ≥ nghiệm tốt nhất hiện có (upper bound)

5.3.2. Cận dưới: Lagrangian Relaxation và Assignment Problem

Một cận dưới mạnh cho TSP được tính bằng cách giải **bài toán gán (Assignment Problem)** — tìm tập hợp cạnh trọng số nhỏ nhất sao cho mỗi nút có đúng một cung vào và một cung ra. AP có thể giải trong $O(n^3)$ bằng thuật toán Hungarian. Kết quả AP là cận dưới cho TSP vì: mọi chu trình Hamilton cũng là một bài giải của AP.

Cận dưới mạnh hơn nữa được tính bằng **minimum spanning tree (MST)**: tổng trọng số MST ≤ hành trình tối ưu (vì bỏ một cạnh của chu trình tối ưu cho MST). Cận dưới này được dùng trong nhiều solver hiện đại.

5.3.3. Hạn chế

B&B trong trường hợp xấu nhất vẫn có độ phức tạp hàm mũ. Tuy nhiên, trong thực tế với bài toán có cấu trúc tốt, B&B thường cắt tỉa được phần lớn cây tìm kiếm và hiệu quả hơn nhiều so với vét cạn. Kết hợp

với các heuristic khởi tạo nghiệm ban đầu tốt, B&B có thể giải được TSP với vài trăm đến vài nghìn thành phố.

5.4. Linear Programming và Cutting Plane

5.4.1. Công thức LP cho TSP

TSP có thể được công thức hóa như một bài toán quy hoạch nguyên (Integer Programming). Gọi $x_{ij} \in \{0, 1\}$ là biến chỉ báo: $x_{ij} = 1$ nếu cạnh (i,j) được chọn. TSP tương đương với:

$$\text{Minimize } \Sigma_{ij} d_{ij} x_{ij}$$

$$\text{Subject to: } \Sigma_j x_{ij} = 1 \ \forall i \text{ (mỗi thành phố được rời đi một lần)}$$

$$\Sigma_i x_{ij} = 1 \ \forall j \text{ (mỗi thành phố được đến đúng một lần)}$$

$$\Sigma_{i,j \in S} x_{ij} \leq |S| - 1 \ \forall S \subset V, S \neq \emptyset \text{ (ràng buộc loại vòng lặp con)}$$

Ràng buộc cuối (subtour elimination constraints) là cốt lõi — không có chúng, bài toán có thể cho kết quả gồm nhiều vòng lặp nhỏ thay vì một chu trình Hamilton. Số lượng ràng buộc này là $2^n - 2$ — quá nhiều để thêm vào cùng một lúc.

5.4.2. Cutting Plane Method

Giải pháp: bắt đầu với các ràng buộc dễ, sau đó thêm dần ràng buộc cắt mặt phẳng (cutting planes) khi phát hiện vi phạm. Phương pháp này, được phát triển bởi Gomory và được hoàn thiện bởi Padberg-Rinaldi, cho phép giải các bài toán TSP rất lớn. **Concorde TSP Solver** (Applegate, Bixby, Chvátal, Cook) sử dụng phương pháp này kết hợp với B&B và đã giải được bài toán 85,900 thành phố.

CHƯƠNG 6: THUẬT TOÁN XẤP XỈ VÀ HEURISTIC

6.1. Heuristic xây dựng (Construction Heuristics)

6.1.1. Nearest Neighbor Heuristic

Thuật toán đơn giản và nhanh nhất: bắt đầu từ một thành phố, luôn di chuyển đến thành phố gần nhất chưa được thăm. Độ phức tạp: $O(n^2)$. Thời gian chạy rất nhanh nhưng chất lượng thường kém, có thể tệ hơn tối ưu tới 20-25%.

Pseudocode: Nearest Neighbor

```
function NearestNeighbor_TSP(D, start):
    visited ← {start}; tour ← [start]; current ← start
    while |visited| < n:
        next ← argmin_{j ∉ visited} D[current][j]
        visited.add(next); tour.append(next); current ← next
    tour.append(start)  // Return to start
    return tour
```

6.1.2. Greedy Algorithm

Greedy khác Nearest Neighbor ở chỗ: thay vì mở rộng từ một điểm hiện tại, ta sắp xếp tất cả cạnh theo trọng số tăng dần và thêm vào tour nếu không vi phạm điều kiện Hamilton (không tạo nhánh, không tạo vòng nhỏ sớm). Thường cho kết quả tốt hơn Nearest Neighbor một chút.

6.1.3. Christofides Algorithm

Đây là thuật toán xấp xỉ với **bảo đảm lý thuyết tốt nhất** cho metric TSP: kết quả không tệ hơn 3/2 lần tối ưu. Thuật toán gồm 5 bước:

3. **Tìm MST:** Tìm cây khung nhỏ nhất T của đồ thị G
4. **Tìm đỉnh bậc lẻ:** O = tập tất cả đỉnh có bậc lẻ trong T (|O| luôn chẵn)
5. **Matching hoàn hảo tối thiểu:** Tìm perfect matching M trên O với tổng trọng số nhỏ nhất
6. **Kết hợp:** Ghép T và M thành multigraph H (mọi đỉnh có bậc chẵn)
7. **Euler tour → Hamilton:** Tìm chu trình Euler trong H, rút gọn thành chu trình Hamilton bằng shortcutting

Phân tích bảo đảm: Gọi OPT là chi phí tối ưu. Ta có: L(MST) ≤ OPT (bỏ một cạnh của chu trình tối ưu = spanning tree). L(M) ≤ OPT/2 (matching nửa của chu trình tối ưu trên O). Do đó: L(Christofides) ≤ L(T) + L(M) ≤ OPT + OPT/2 = 3·OPT/2. ■

Cập nhật 2022: Thuật toán Karlin-Klein-Gharan
Năm 2022, Karlin, Klein và Oveis Gharan công bố thuật toán đầu tiên vượt qua rào cản 3/2 của Christofides sau 46 năm, đạt hệ số xấp xỉ 3/2 - ε với $\varepsilon > 10^{-36}$. Đây là bước đột phá lý thuyết quan trọng, mặc dù cải tiến rất nhỏ về mặt thực tế.

6.2. Local Search — Tối ưu hóa cục bộ

6.2.1. 2-opt

2-opt là kỹ thuật cải thiện tour phổ biến nhất. Ý tưởng: xóa 2 cạnh không giao nhau khỏi tour và nối lại theo cách khác. Nếu tour ngắn hơn, chấp nhận. Lặp đến khi không cải thiện được nữa (2-opt local optimal). Độ phức tạp mỗi bước: $O(n^2)$. Thực tế: cải thiện đáng kể so với Nearest Neighbor, thường trong vòng 5% tối ưu cho bài toán ngẫu nhiên.

6.2.2. 3-opt và k-opt

Tổng quát hóa 2-opt: xóa k cạnh và nối lại. k-opt cho kết quả tốt hơn nhưng phức tạp hơn: 3-opt có $O(n^3)$ cặp, k-opt có $O(n^k)$. Trong thực tế, 3-opt là thường dùng nhất; k > 3 hiếm khi được triển khai đầy đủ.

6.2.3. Lin-Kernighan Algorithm (LK)

Lin-Kernighan (1973) là thuật toán local search mạnh nhất và linh hoạt nhất cho TSP. Không cố định k, LK thực hiện các bước hoán đổi cạnh động — có thể là 2-opt, 3-opt, hoặc cao hơn tùy theo bài toán cụ thể. LK-Helsgott (LKH) là phiên bản hiện đại do Helsgott (2000, 2012) phát triển, sử dụng penalty function và candidate lists để tăng hiệu quả. LKH được coi là **solver thực tế tốt nhất cho TSP** — thường tìm được nghiệm trong vòng 1% so với tối ưu trong thời gian hợp lý.

6.3. Thuật toán Metaheuristic

6.3.1. Simulated Annealing (SA)

Được lấy cảm hứng từ quá trình luyện kim (ủ và làm nguội kim loại), SA (Kirkpatrick et al., 1983) cho phép chấp nhận nghiệm xấu hơn với xác suất P = e^(-ΔE/T) trong đó ΔE là mức tăng chi phí và T là 'nhiệt

độ'. Khi T giảm dần theo lịch làm nguội (cooling schedule), thuật toán trở nên ít phiêu lưu hơn và hội tụ về optima cục bộ tốt.

Tham số	Ý nghĩa	Giá trị điển hình
T_0 (nhiệt độ ban đầu)	Mức chấp nhận xáo trộn ban đầu	Đủ lớn để chấp nhận 80% nghiệm xấu
α (cooling rate)	Tốc độ làm nguội	0.90 - 0.99
T_{min} (nhiệt độ dừng)	Điều kiện dừng	$0.001 \times T_0$
L (độ dài Markov chain)	Số bước mỗi nhiệt độ	n đến n^2

Bảng 6.1: Tham số Simulated Annealing cho TSP

6.3.2. Genetic Algorithm (GA)

GA mô phỏng tiến hóa sinh học: duy trì một quần thể (population) các tour, áp dụng các toán tử **selection** (chọn lọc), **crossover** (lai ghép), và **mutation** (đột biến) để tạo ra các thế hệ tour tốt hơn dần. Thách thức đặc biệt của GA cho TSP: thiết kế toán tử crossover không phá vỡ cấu trúc hoán vị hợp lệ. Các toán tử phổ biến: Order Crossover (OX), Partially Mapped Crossover (PMX), Cycle Crossover (CX).

6.3.3. Ant Colony Optimization (ACO)

Được lấy cảm hứng từ hành vi tìm đường của kiến, ACO (Dorigo, 1992) sử dụng **pheromone trails** — một bộ nhớ tập thể — để hướng dẫn tìm kiếm. Mỗi con kiến nhân tạo xây dựng một tour ngẫu nhiên theo quy tắc: xác suất chọn thành phố j từ thành phố i tỉ lệ với $\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$ trong đó τ_{ij} là mức pheromone (học từ lịch sử tốt), $\eta_{ij} = 1/d_{ij}$ là thông tin heuristic, α và β là tham số cân bằng. Sau đó pheromone được cập nhật: tăng trên các cạnh của tour tốt, giảm qua bay hơi (evaporation).

6.3.4. Particle Swarm Optimization (PSO)

PSO (Kennedy & Eberhart, 1995) mô phỏng hành vi bầy (swarm behavior) của chim hoặc cá. Mỗi hạt (particle) đại diện cho một tour, di chuyển trong không gian giải pháp dưới ảnh hưởng của inertia (quán tính), cognitive component (hướng về vị trí tốt nhất cá nhân), và social component (hướng về vị trí tốt nhất toàn bộ bầy). PSO gặp khó khăn hơn với TSP do không gian tổ hợp rời rạc.

6.3.5. Tabu Search (TS)

Tabu Search (Glover, 1986) sử dụng một danh sách cấm (tabu list) để ngăn tìm kiếm quay lại các nghiệm vừa thăm, giúp thoát khỏi tối ưu cục bộ. TS có thể chấp nhận bước tệ hơn nếu không vi phạm tabu. Cơ

chế **aspiration criteria** cho phép phá quy tắc tabu nếu kết quả đủ tốt. Tabu Search rất hiệu quả cho TSP và nhiều biến thể.

6.4. Neural Networks và Deep Learning

6.4.1. Pointer Networks (PtrNet)

Vinyals et al. (2015) đề xuất Pointer Networks, một biến thể của sequence-to-sequence với attention mechanism, trong đó đầu ra là một 'con trỏ' chỉ vào một phần tử của đầu vào. Mô hình được huấn luyện với supervision (biết nghiệm tối ưu) để học ánh xạ từ tập thành phố → hành trình. Trên TSP nhỏ ($n \leq 50$), PtrNet cho kết quả cạnh tranh với các heuristic truyền thống.

6.4.2. Attention Model / Transformer-based

Kool et al. (2019) đề xuất Attention Model với Reinforcement Learning (không cần biết nghiệm tối ưu), sử dụng kiến trúc Transformer với multi-head attention. Mô hình học policy giải TSP bằng cách nhận thưởng dựa trên chất lượng tour. Kết quả: vượt qua S2V-DQN và gần bằng LKH-3 cho $n \leq 100$, với tốc độ suy luận rất nhanh (GPU parallel decoding).

6.4.3. Graph Neural Networks (GNN)

Nhiều công trình gần đây sử dụng GNN để biểu diễn đồ thị thành phố và học các edge embeddings. Kết hợp với beam search hoặc MCTS (Monte Carlo Tree Search), các phương pháp GNN đạt kết quả ấn tượng. Tuy nhiên, tất cả các phương pháp deep learning hiện tại vẫn chưa vượt qua LKH-3 (Lin-Kernighan-Helsgott) cho bài toán lớn ($n > 1000$).

6.5. So sánh tổng hợp các phương pháp

Phương pháp	Loại	Độ phức tạp	Chất lượng	Thực tế
Brute Force	Chính xác	$O(n!)$	Tối ưu 100%	$n \leq 12$
Held-Karp (DP)	Chính xác	$O(n^2 \cdot 2^n)$	Tối ưu 100%	$n \leq 25$
Branch & Bound	Chính xác	Worst $O(n!)$	Tối ưu 100%	$n \leq 100$
Concorde (LP+BnB)	Chính xác	Thực nghiệm tốt	Tối ưu 100%	$n \leq 85{,}900$
Nearest Neighbor	Greedy	$O(n^2)$	~120-125%	Mọi n
Christofides	Xấp xỉ	$O(n^3)$	≤ 150%	$n \leq 100K$
2-opt	Local Search	$O(n^2)$/iter	~105-110%	$n \leq 100K$
LKH-3	Local Search	$\sim O(n^2)$	~101-102%	$n \leq 10M$
SA	Metaheuristic	Cấu hình	~102-105%	$n \leq 1M$

ACO	Metaheuristic	$O(n^2 \cdot iter)$	~103-107%	$n \leq 100K$
Attention Model	Deep Learning	$O(n^2)$	~103-108%	$n \leq 1000$

Bảng 6.2: So sánh tổng hợp các thuật toán TSP (% so với tối ưu)

CHƯƠNG 7: ỨNG DỤNG THỰC TIỄN

7.1. Logistics và Giao thông vận tải

Đây là lĩnh vực ứng dụng lớn nhất và trực tiếp nhất của TSP và VRP. Theo báo cáo của McKinsey (2020), tối ưu hóa tuyến đường có thể giảm chi phí vận chuyển từ 10-30%. Các bài toán điển hình:

- **Giao hàng chặng cuối (Last-mile delivery):** Amazon, FedEx, UPS, Grab, Lalamove — lên lịch tuyến đường cho hàng trăm tài xế giao hàng đến hàng nghìn điểm mỗi ngày. Đây là CVRP với cửa sổ thời gian.
- **Lập kế hoạch tuyến xe buýt/metro:** Tối ưu hóa tuyến đường để phủ sóng tối đa điểm dân cư với số xe giới hạn.
- **Xe chở học sinh:** School Bus Routing Problem — một biến thể CVRP phức tạp với ràng buộc thời gian.
- **Tàu biển / container:** Lập kế hoạch hành trình tàu biển qua nhiều cảng để tối thiểu hóa chi phí nhiên liệu.

7.2. Sản xuất và Gia công

7.2.1. Thiết kế vi mạch điện tử (VLSI)

Trong thiết kế VLSI (Very Large Scale Integration), máy khoan hoặc máy hàn phải đi qua hàng nghìn điểm trên bo mạch. Thứ tự di chuyển ảnh hưởng trực tiếp đến thời gian sản xuất. Đây là ứng dụng TSP điển hình, thường với n từ vài nghìn đến vài chục nghìn điểm. Bài toán TSP 85,900 thành phố được giải bởi Applegate et al. chính là từ dữ liệu thực tế của **bản đồ thụy điển (Sweden PCB drilling problem).**

7.2.2. Lập lịch máy CNC

Máy cắt CNC phải gia công nhiều điểm trên vật liệu. Tối ưu hóa thứ tự gia công (TSP) giúp giảm hành trình di chuyển không cần thiết, tăng tốc độ sản xuất và tiết kiệm năng lượng.

7.3. Sinh học và Y học

7.3.1. Giải trình tự DNA (DNA Sequencing)

Trong giải trình tự DNA, một bài toán quan trọng là **Sequencing by Hybridization (SBH)**: cho tập các chuỗi DNA ngắn (k-mers), tìm chuỗi DNA dài nhất chứa tất cả k-mers. Điều này tương đương với tìm Hamiltonian path trong đồ thị de Bruijn — một dạng TSP. Kỹ thuật này được dùng trong HGP (Human Genome Project).

7.3.2. Protein Folding

Dự đoán cấu trúc protein có liên quan đến bài toán tối ưu hóa tổ hợp tương tự TSP. Việc tìm chuỗi amino acid nào dẫn đến cấu trúc 3D ổn định nhất là một bài toán NP-Hard.

7.3.3. Lập lịch phẫu thuật

Trong bệnh viện, lập lịch các ca phẫu thuật và phân bổ phòng mổ có thể được mô hình hóa như TSP với nhiều ràng buộc thời gian và nguồn lực.

7.4. Thiên văn học và Khoa học

7.4.1. Quan sát kính thiên văn

Kính thiên văn phải chỉ vào nhiều mục tiêu quan sát trong một đêm. Việc tối ưu hóa thứ tự quan sát (minimizing slew time — thời gian xoay kính) là một bài toán TSP với ràng buộc về khả năng nhìn thấy (visibility windows) theo thời gian. NASA sử dụng thuật toán TSP cho lập lịch Hubble Space Telescope.

7.4.2. Phân tích phổ (Spectroscopy)

Trong phân tích phổ học, thiết bị đo cần di chuyển qua nhiều điểm đo theo thứ tự tối ưu để giảm thời gian di chuyển.

7.5. Robotics và Tự động hóa

7.5.1. Coverage Path Planning

Robot lau nhà (Roomba), máy bay không người lái (drone) phun thuốc nông nghiệp, hoặc robot kiểm tra nhà máy cần thực hiện **coverage path planning** — lập kế hoạch hành trình để phủ sóng toàn bộ khu vực. Bài toán này liên quan chặt chẽ đến TSP và Chinese Postman Problem.

7.5.2. Pick-and-Place trong kho hàng

Trong kho hàng tự động (như của Amazon Robotics), robot phải lấy nhiều mặt hàng từ nhiều vị trí. Thứ tự lấy hàng ảnh hưởng trực tiếp đến tổng khoảng cách di chuyển — đây là TSP trong không gian kho hàng.

7.6. Mạng máy tính và Viễn thông

Trong thiết kế mạng truyền thông, bài toán tìm đường dây kết nối các trạm phát sóng hoặc trạm truy cập với chi phí tối thiểu tương tự TSP. Đặc biệt, bài toán **Min-cost Hamiltonian Path** xuất hiện trong thiết kế mạng backbone và cáp quang.

7.7. Tài chính

Trong quản lý danh mục đầu tư, bài toán tối ưu hóa thứ tự giao dịch để giảm chi phí giao dịch và tác động thị trường có thể được mô hình hóa như TSP. Đặc biệt, trong **high-frequency trading** (giao dịch tần số cao), tối ưu hóa thứ tự thực hiện lệnh là bài toán TSP thời gian thực.

CHƯƠNG 8: HƯỚNG NGHIÊN CỨU MỞ VÀ TƯƠNG LAI

8.1. Các bài toán mở trong lý thuyết

8.1.1. Bài toán P vs NP

Câu hỏi liệu P = NP hay P ≠ NP vẫn là bài toán mở quan trọng nhất trong khoa học máy tính. Nếu được giải quyết theo hướng P = NP, điều đó sẽ có ý nghĩa cách mạng đối với TSP và toàn bộ lý thuyết tính toán. Nếu P ≠ NP được chứng minh chặt chẽ, các nhà nghiên cứu sẽ có nền tảng vững chắc hơn để thiết kế các thuật toán xấp xỉ và heuristic.

8.1.2. Cải thiện bảo đảm xấp xỉ

Sau kết quả đột phá của Karlin-Klein-Gharan (2022) phá vỡ rào cản 3/2 của Christofides, câu hỏi đặt ra là: giới hạn dưới thực sự của hệ số xấp xỉ tốt nhất có thể đạt được cho metric TSP là bao nhiêu? Liệu có thể đạt được hệ số $(1 + \varepsilon)$ cho mọi $\varepsilon > 0$ (PTAS)? Câu hỏi này vẫn còn mở.

8.1.3. Cấu trúc bài toán khó nhất

Trường hợp nào của TSP là khó nhất? Các bài toán ngẫu nhiên thường dễ hơn bài toán cấu trúc xấu nhất. Việc xác định các trường hợp 'worst case' cụ thể và hiểu cấu trúc của chúng là hướng nghiên cứu quan trọng.

8.2. Điện toán lượng tử và TSP

8.2.1. Thuật toán Grover

Thuật toán Grover (1996) cung cấp tăng tốc bậc hai (quadratic speedup) cho bài toán tìm kiếm không có cấu trúc: tìm kiếm trong N phần tử chỉ cần $O(\sqrt{N})$ bước lượng tử thay vì $O(N)$ bước cổ điển. Áp dụng vào TSP brute force, Grover giảm thời gian từ $O((n-1)!)$ xuống $O(\sqrt{(n-1)!})$ — vẫn là siêu đa thức, không giải được bài toán lớn.

8.2.2. Quantum Annealing

D-Wave Systems đã triển khai máy tính lượng tử annealing với hàng nghìn qubit, và đây được quảng bá như một cách tiếp cận tự nhiên cho TSP và các bài toán tối ưu hóa tổ hợp. Tuy nhiên, kết quả thực nghiệm hiện tại còn hỗn hợp — quantum annealing chưa thể vượt qua các solver cổ điển tốt nhất cho TSP quy mô lớn.

8.2.3. QAOA (Quantum Approximate Optimization Algorithm)

QAOA (Farhi et al., 2014) là thuật toán lượng tử lai (hybrid quantum-classical) cho tối ưu hóa tổ hợp. QAOA xây dựng trạng thái lượng tử thông qua p lớp xoay cổng, tối ưu hóa các tham số bằng thuật toán cổ điển. Cho TSP nhỏ, QAOA cho thấy tiềm năng nhưng vẫn còn hạn chế bởi noise trong phần cứng lượng tử hiện tại (NISQ era).

8.3. Học máy và AI hiện đại

8.3.1. Large Language Models (LLM) và TSP

Có nhiều thí nghiệm gần đây thử nghiệm GPT-4 và các LLM trong việc giải TSP. Kết quả cho thấy LLM không thể giải TSP hiệu quả từ đầu — chúng không có khả năng tổ hợp số học cần thiết. Tuy nhiên, LLM có thể hỗ trợ trong việc thiết kế thuật toán, diễn giải kết quả, và kết hợp với solver truyền thống.

8.3.2. Neural Combinatorial Optimization (NCO)

NCO là lĩnh vực đang phát triển nhanh, kết hợp Deep Learning với các bài toán tối ưu hóa tổ hợp. Các hướng chính hiện tại: **(1) Constructive methods** (xây dựng tour từng bước với RL), **(2) Improvement methods** (học cách cải thiện tour hiện có), **(3) Hybrid methods** (kết hợp neural với solver truyền thống như LKH). Các mô hình DPDP (Dynamic Programming with Deep Learning), DIFUSCO (2023) sử dụng Diffusion Model, và các phương pháp GNN ngày càng cạnh tranh với solver truyền thống.

8.3.3. Generalization Challenge

Thách thức lớn nhất của NCO là **generalization** — khả năng tổng quát hóa sang bài toán lớn hơn và phân phối khác với dữ liệu huấn luyện. Hầu hết các mô hình hiện tại được huấn luyện trên bài toán với n cố định và phân phối ngẫu nhiên đồng đều, hoạt động kém khi n lớn hơn hoặc phân phối thay đổi.

8.4. Hướng nghiên cứu ứng dụng

8.4.1. Real-time TSP

Bài toán TSP thời gian thực với ràng buộc nghiêm ngặt về thời gian (vài giây để tìm nghiệm) ngày càng quan trọng trong logistics on-demand, xe tự lái, và hệ thống giao hàng thông minh. Hướng nghiên cứu này cần cân bằng giữa chất lượng nghiệm và tốc độ tính toán.

8.4.2. Multi-objective TSP

Thực tế thường yêu cầu tối ưu hóa nhiều mục tiêu đồng thời: giảm khoảng cách VÀ thời gian VÀ phát thải carbon VÀ tải trọng cân bằng giữa các xe. Multi-objective TSP (MO-TSP) cần tìm Pareto front thay vì một nghiệm duy nhất.

8.4.3. Green TSP

Green Logistics đang trở thành ưu tiên toàn cầu. Green TSP tối ưu hóa lượng phát thải CO_2 hoặc tiêu thụ nhiên liệu thay vì khoảng cách thuần túy. Mô hình phức tạp hơn vì chi phí nhiên liệu phụ thuộc vào tải trọng, tốc độ, và loại đường.

8.5. Tóm tắt hướng nghiên cứu

Hướng nghiên cứu	Trạng thái	Tiềm năng	Thách thức
Quantum Computing	Sơ khai (NISQ era)	Cao dài hạn	Noise, qubit count
Neural NCO	Đang phát triển nhanh	Rất cao	Generalization
Improved Approx. Ratio	Tiến triển chậm	Lý thuyết cao	Khó về lý thuyết
P vs NP	Không có tiến bộ	Cách mạng	Cực kỳ khó
Real-time solver	Đang phát triển	Ứng dụng cao	Cân bằng QoS
Multi-objective TSP	Đang phát triển	Thực tiễn cao	Phức tạp hóa

Bảng 8.1: Tổng hợp hướng nghiên cứu TSP

CHƯƠNG 9: KẾT LUẬN

9.1. Tổng hợp các phát hiện chính

Báo cáo này đã trình bày một phân tích toàn diện về bài toán Người đi du lịch (TSP) và các vấn đề liên quan. Dưới đây là các phát hiện và kết luận chính:

9.1.1. Về bản chất của bài toán

- **TSP là NP-Hard** — không tồn tại thuật toán đa thức nào (theo hiểu biết hiện tại) có thể luôn tìm ra hành trình tối ưu
- **Không gian giải pháp tăng giai thừa** — với n thành phố, có (n-1)!/2 hành trình phân biệt, làm cho vét cạn hoàn toàn bất khả thi ngay cả với n=25
- **TSP tổng quát không thể xấp xỉ** — không có bảo đảm xấp xỉ hữu hạn nếu P ≠ NP; chỉ TSP Metric mới có bảo đảm

9.1.2. Về trạng thái của thuật toán

- **Thuật toán chính xác** giới hạn ở n ≤ 85,900 (kỷ lục của Concorde), không thực tế cho bài toán n > 1,000 thông thường
- **LKH-3** là thuật toán thực tế tốt nhất — thường tìm được nghiệm trong vòng 1% tối ưu cho n lên đến hàng triệu thành phố
- **Neural NCO** có tiềm năng lớn nhưng chưa vượt qua LKH-3 cho bài toán lớn; thách thức chính là generalization

9.1.3. Về tương lai

- **P vs NP** vẫn là câu hỏi mở quan trọng nhất; hầu hết chuyên gia tin P ≠ NP
- **Máy tính lượng tử** chưa thể vượt qua solver cổ điển tốt nhất hiện nay cho TSP quy mô thực tế
- **Hướng thực tiễn** là kết hợp AI/ML với solver truyền thống để tạo ra các hybrid solver mạnh hơn

9.2. Trả lời câu hỏi trung tâm

> *Tại sao chúng ta không thể tìm ra lời giải tối ưu cho TSP với số lượng thành phố lớn trong thời gian thực?*
>
> **— Câu hỏi trung tâm**

Câu trả lời ngắn gọn: Vì TSP là NP-Hard — một lớp bài toán mà lý thuyết tính toán cho thấy (có điều kiện) không thể giải hiệu quả bằng bất kỳ thuật toán cổ điển nào.

Câu trả lời đầy đủ: Có ba nguyên nhân cơ bản:

8. **Rào cản tổ hợp:** Số hành trình cần xét là (n-1)!/2 — tăng siêu hàm mũ. Với n=25, thời gian vét cạn đã là hàng triệu năm trên siêu máy tính hiện đại. Không có phím tắt toán học nào cho phép tìm tối ưu nhanh hơn cơ bản.
9. **Rào cản lý thuyết:** TSP là NP-Hard — được chứng minh bằng cách quy đổi từ Hamiltonian Cycle. Theo giả thuyết P ≠ NP (được hầu hết nhà khoa học tin là đúng), không tồn tại thuật toán đa thức nào cho TSP. Câu hỏi P vs NP là một trong bảy Bài toán Thiên niên kỷ chưa được giải.
10. **Rào cản thực tế:** Ngay cả thuật toán DP tốt nhất (Held-Karp, $O(n^2 \cdot 2^n)$) cũng yêu cầu bộ nhớ và thời gian hàm mũ, giới hạn ở $n \leq 30$. Các bài toán thực tế với n=1,000 đến n=1,000,000 chỉ có thể giải xấp xỉ.

9.3. Thông điệp cuối

TSP không chỉ là một bài toán toán học khô khan — nó là biểu tượng cho một trong những câu hỏi sâu sắc nhất của khoa học: **"Ranh giới của tính toán là gì?"** TSP nhắc nhở chúng ta rằng ngay cả những bài toán có thể phát biểu trong một câu cũng có thể ẩn chứa độ phức tạp vô tận. Và đó chính là vẻ đẹp của toán học và khoa học máy tính lý thuyết.

> *Tôi có thể tính được chuyển động của các thiên thể, nhưng không thể tính được sự điên rồ của loài người.*
>
> **— Isaac Newton — (và có lẽ cũng không thể giải TSP với 100 thành phố)**

TÀI LIỆU THAM KHẢO

Sách và Chuyên khảo

- Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Garey, M. R., & Johnson, D. S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman.
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1985). The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization. Wiley.
- Papadimitriou, C. H. (1994). Computational Complexity. Addison-Wesley.

Bài báo Khoa học Quan trọng

- Bellman, R. (1962). Dynamic programming treatment of the travelling salesman problem. Journal of the ACM, 9(1), 61-63.
- Christofides, N. (1976). Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Graduate School of Industrial Administration, Carnegie Mellon University.
- Cook, S. A. (1971). The complexity of theorem-proving procedures. STOC 1971, 151-158.
- Dantzig, G., Fulkerson, R., & Johnson, S. (1954). Solution of a large-scale traveling-salesman problem. Journal of the Operations Research Society of America, 2(4), 393-410.
- Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics, 26(1), 29-41.
- Helsgott, K. (2000). An effective implementation of the Lin–Kernighan traveling salesman heuristic. European Journal of Operational Research, 126(1), 106-130.
- Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. Journal of the Society for Industrial and Applied Mathematics, 10(1), 196-210.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In Complexity of Computer Computations (pp. 85-103). Springer.

- Karlin, A., Klein, N., & Gharan, S. O. (2022). A (slightly) improved approximation algorithm for metric TSP. STOC 2021.
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671-680.
- Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! ICLR 2019.
- Lin, S., & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. Operations Research, 21(2), 498-516.
- Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. Advances in Neural Information Processing Systems, 28.

Công cụ và Phần mềm

- Concorde TSP Solver: http://www.math.uwaterloo.ca/tsp/concorde/
- LKH-3 Solver (Helsgott): http://webhotel4.ruc.dk/~keld/research/LKH-3/
- TSPLIB Benchmark: http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/
- OR-Tools (Google): https://developers.google.com/optimization/routing/tsp

Tài nguyên Trực tuyến

- P vs NP Problem — Clay Mathematics Institute: https://www.claymath.org/millennium-problems/p-vs-np-problem
- NIST Handbook of Mathematical Functions — Complexity Theory Primer
- The TSP Homepage: https://www.math.uwaterloo.ca/tsp/

PHỤ LỤC

Phụ lục A: Ví dụ tính toán Held-Karp

Minh họa với n = 4 thành phố (1, 2, 3, 4) và ma trận khoảng cách:

	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

Ma trận khoảng cách ví dụ 4 thành phố

Bước 1 — Trường hợp cơ sở (tập S chứa một phần tử):

- $C(\{2\}, 2) = d_{12} = 10$
- $C(\{3\}, 3) = d_{13} = 15$
- $C(\{4\}, 4) = d_{14} = 20$

Bước 2 — Tập S có 2 phần tử:

- $C(\{2,3\}, 3) = C(\{2\}, 2) + d_{23} = 10 + 35 = 45$
- $C(\{2,3\}, 2) = C(\{3\}, 3) + d_{32} = 15 + 35 = 50$
- $C(\{2,4\}, 4) = C(\{2\}, 2) + d_{24} = 10 + 25 = 35$
- $C(\{2,4\}, 2) = C(\{4\}, 4) + d_{42} = 20 + 25 = 45$
- $C(\{3,4\}, 4) = C(\{3\}, 3) + d_{34} = 15 + 30 = 45$
- $C(\{3,4\}, 3) = C(\{4\}, 4) + d_{43} = 20 + 30 = 50$

Bước 3 — Tập S có 3 phần tử:

- $C(\{2,3,4\}, 4) = \min(C(\{2,3\}, 3)+d_{34}, C(\{2,3\}, 2)+d_{24}) = \min(45+30, 50+25) = \min(75, 75) = 75$
- $C(\{2,3,4\}, 3) = \min(C(\{2,4\}, 4)+d_{43}, C(\{2,4\}, 2)+d_{23}) = \min(35+30, 45+35) = \min(65, 80) = 65$
- $C(\{2,3,4\}, 2) = \min(C(\{3,4\}, 4)+d_{42}, C(\{3,4\}, 3)+d_{32}) = \min(45+25, 50+35) = \min(70, 85) = 70$

Bước 4 — Nghiệm tối ưu:

OPT = min(C({2,3,4}, 4)+d_{41}, C({2,3,4}, 3)+d_{31}, C({2,3,4}, 2)+d_{21}) = min(75+20, 65+15, 70+10) = min(95, 80, 80) = **80**

Hành trình tối ưu: 1 → 3 → 4 → 2 → 1 (hoặc 1 → 2 → 4 → 3 → 1) với tổng khoảng cách = 15 + 30 + 25 + 10 = 80.

Phụ lục B: Bảng ký hiệu

Ký hiệu	Ý nghĩa
n	Số lượng thành phố (đỉnh)
G = (V, E, w)	Đồ thị có trọng số
V	Tập đỉnh (thành phố)
E	Tập cạnh (khoảng cách)
w hoặc d	Hàm trọng số (khoảng cách)
π	Hoán vị (hành trình)
L(π)	Tổng độ dài hành trình π
OPT	Nghiệm tối ưu (độ dài nhỏ nhất)
C(S, i)	Hàm Held-Karp: đường đi qua S, kết thúc tại i
P	Lớp bài toán giải được trong thời gian đa thức
NP	Lớp bài toán xác minh được trong thời gian đa thức
$\leq_p$	Quy đổi đa thức (polynomial reduction)

Bảng B.1: Bảng ký hiệu toán học